

CS492 Fall 2026 Unit Exam 1 Review

Sebastian Sztolberg

October 8, 2026

1 Intro to MMIO

Given

- UART peripheral mapped to physical memory via MMIO
- 16-bit address-space
- Base address: 0x1000
- Each register is mapped to 1 byte
- MMIO operations take 10 cycles
- All other operations take 1 cycle

Register Specification

Offset	Register	Description
+0x00	status	Status register
+0x01	tx_data	Transmit to port
+0x02	rx_data	Receive from port

Status Register Bit Masks

Mask	Meaning
0x01	tx_data empty
0x02	rx_data ready

Bit Mask Example

Value	Binary
Status register (s)	01010011
tx_data mask (t)	00000001
$s \& t$	00000001

The result extracts the single bit representing whether `tx_data` is empty and ready for transmission.

- Nonzero result $\rightarrow$ `tx_data` is empty and ready to write
- Zero result $\rightarrow$ `tx_data` is not empty and not ready to write

Code

```
1 int main(void) {
2     // Declared volatile to prevent compiler optimizations (we always want
3     // it to read/write to memory)
4     volatile uint8_t * restrict status = (volatile uint8_t *)0x1000;
5     volatile uint8_t * restrict tx_data = (volatile uint8_t *)0x1001;
6
7     char c[1] = "H";
8     while(!(*status & 0x01));
9     *tx_data = *(volatile uint8_t *)(c);
10
11     exit(EXIT_SUCCESS);
12 }
```

Corresponding Assembly

```
1     LDR r0, =0x1000      @ Load base address into register r0
2     LDR r1, =c           @ Load character address into register r1
3
4 while_loop:
5     LDR r2, [r0]         @ Load current status into register r2
6     AND r2, r2, 0x01     @ Apply TX_data empty bitmask to status
7     CMP r2, #0           @ Check if TX_data is empty and ready for write
8     BEQ while_Loop      @ Loop if not empty
9
10    LDR r3, [r1]          @ Load the character into register r3
11    STR r3, [r0, #1]      @ Store the character in the TX_data register
12
13    RET                  @ Return from the function call
```

Questions

1. How many cycles does the following program body take?
2. Given the fact that the CPU can process 200k cycles a second, and a full write takes 200 cycles, how many write operations do we need to do to achieve 20 percent CPU utilization

2 MMIO and Cycle-Counting

Given

Assume the following specifications:

- UART peripheral mapped to physical memory (MMIO)
- 32-bit address-space
- Base address: 0x10000000
- Each register allocated 4 bytes
- Data may only be transmitted/received **1 byte** at a time
- MMIO operations take 10 cycles, all other operations take 1 cycle

Register Specification

Offset	Register	Description
+0x00	status	Status register
+0x04	tx_data	Transmit to port
+0x08	rx_data	Receive from port

Status Register Bit Masks

Mask	Meaning
0x01	tx_data empty
0x02	rx_data ready

Code

```
1 #define UART_STATUS_TX_EMPTY_MASK 0x01
2 #define UART_STATUS_RX_READY_MASK 0x02
3
4 int main(void) {
5     volatile uint32_t * uart_status =
6         (volatile uint32_t*)(0x10000000);
7
8     volatile uint32_t * uart_tx_data =
9         (volatile uint32_t*)(0x10000000 + 0x04);
10
11     volatile uint32_t * uart_rx_data =
12         (volatile uint32_t*)(0x10000000 + 0x08);
13
14     uint8_t w[8];
15
16     for (int i = 0; i < 8; i++) {
17         while((*uart_status & UART_STATUS_RX_READY_MASK) == 0);
18         w[i] = *(uint8_t*)(uart_rx_data);
19     }
20
21     for (int j = 0; j < 8; j++) {
22         while((*uart_status & UART_STATUS_TX_EMPTY_MASK) == 0);
23         *uart_tx_data = *(volatile uint32_t*)(w[j]);
24     }
25
26     return 0;
27 }
28
29 }
```

Corresponding Function Body Assembly:

```
1     LDR r1, =0x10000000    @ base address
2     LDR r2, =w             @ word array address
3
4     MOV r0, #8             @ loop 1 counter
5
6 for_loop1:
7 while_loop1:
8     LDR r3, [r1]
9     AND r3, r3, 0x02
10    CMP r3, #0
11    BEQ while_loop1
12
13    LDRB r3, [r1, #8]
14    STRB r3, [r2]
15
16    ADD r2, r2, #1
17    SUB r0, r0, #1
18    CMP r0, #0
19    BNE for_loop1
20
21
22    LDR r2, =w             @ reload word arr base addr
23    MOV r0, #8             @ reset loop counter
24
25
26 for_loop2:
27 while_loop2:
28     LDR r3, [r1]           @ load status register
29     AND r3, r3, 0x01       @ apply tx_data bitmask
30     CMP r3, #0             @ cmp
31     BEQ while_loop2        @ keep iterating if not ready
32
33     LDRB r3, [r2]          @ read w[i]
34     STRB r3, [r1, #4]      @ store in tx_data
35
36     ADD r2, r2, #1
37     SUB r0, r0, #1
38     CMP r0, #0
39     BNE for_loop2
```

Questions

1. What is the program doing?
2. Under the assumption that both while loops take the same amount of cycles, will both for loops take the same amount of cycles? Show your work.

3. If for each for-loop iteration, the while loop iterates 10 times, how many cycles will it take to complete the function body?

Assume the successful read/write occurs on the 11th cycle.

4. How would we implement a function that constantly waits to receive new data from the port, and then transmits it to a 1-byte software buffer that can be read by some external process? Give C-style pseudocode based on the C code above.

3 Fork Behavior

```
1 int main(int argc, char* argv[]) {
2     pid_t p1;
3     pid_t p2;
4
5     uint8_t x = 0;
6
7     if ((p1 = fork())) {
8         x++;
9     }
10    printf("%u\n", x);
11
12    if (p1 && (p2 = fork())) {
13        x++;
14    }
15    printf("%u\n", x);
16
17    return 0;
18 }
19
20 }
```

Questions

1. Upon a call to `fork()`, what elements of the virtual memory image are shared between the parent and child? Explain.
2. How many processes are there in total? (include the parent)
3. List the prints produced by each process individually

4 Fork, Exec, Wait

```
1 int main(void) {
2     pid_t p1;
3     pid_t p2;
4
5     int n = 0;
6
7     if ((p1 = fork())) {
8         n++;
9     }
10    printf("%d\n", n);
11
12    if (!p1) {
13        p2 = fork();
14        n += 2;
15    }
16
17    if (!p2) {
18        execvp("./helper", NULL);
19    }
20
21    if (p1) {
22        wait(NULL);
23        printf("%d\n", n);
24    } else {
25        wait(NULL);
26        printf("%d\n", n);
27    }
28
29    exit(EXIT_SUCCESS);
30 }
31
32 // helper.c code
33 int main(void) {
34     printf("%d\n", 3);
35     exit(EXIT_SUCCESS);
36 }
```

Questions

1. Explain what `execvp()` and `wait()` do
2. How many processes are spawned during the execution of this program? (include the parent)
3. Please give the independent strings printed by each process
4. Bonus: what do the suffixes of `exec` signify? i.e. `[v, l]` and `[p, e]`

5 PThreads and Thread Shared Resources

```
1 int main(void) {
2     pthread_t t1;
3     pthread_t t2;
4
5     int* np = (int*)malloc(sizeof(int));
6     *np = 0;
7
8
9     pthread_create(&t1, NULL, helper, (void*)np);
10    pthread_create(&t2, NULL, helper, (void*)np);
11
12    pthread_join(t1, NULL);
13    pthread_join(t2, NULL);
14
15    *np += 1;
16
17    printf("%d\n", *np);
18
19    free(np);
20    exit(EXIT_SUCCESS);
21 }
```

1. What are some differences between processes and threads?
2. What values are printed by the program above?
3. We would like to truly parallelize our program. 25 percent of our program is parallelizable. We have 4 CPU cores at our disposal. Please calculate the CPU speedup.